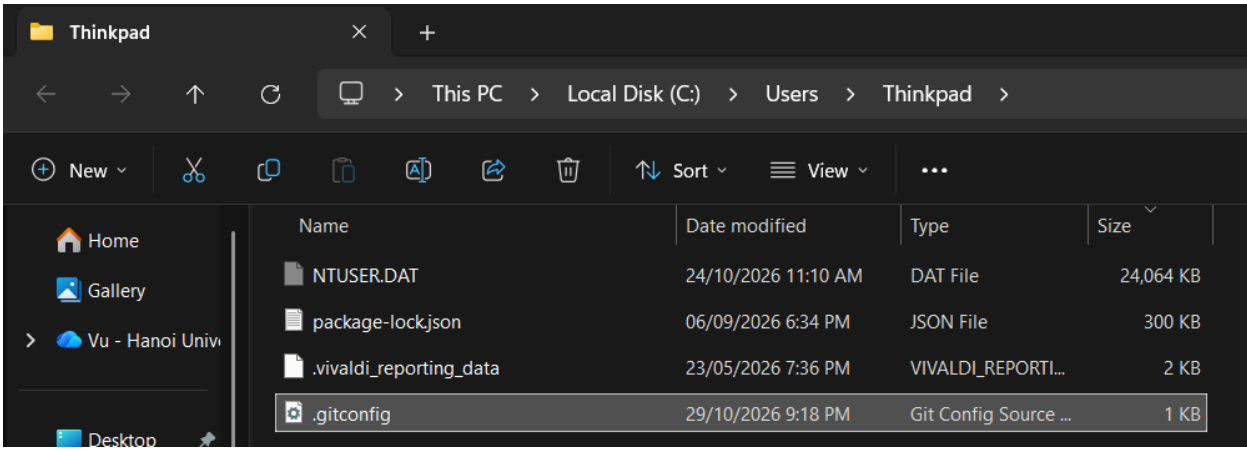
Lab01

- **Câu hỏi nhanh 1**: cấu hình --global được lưu ở đâu? Khác gì so với --local?
  - --global lưu trong ~/.gitconfig (Windows: C:\Users\<tên>\.gitconfig) và áp dụng cho mọi repo của người dùng.
  - 

  - --local lưu trong .git/config của từng repo và chỉ áp dụng cho repo đó.
- **Thảo luận 1**: Vì sao dùng git mv? release/lab01 có phải baseline? Vì sao không commit thẳng master dù làm một mình?
  - Dùng git mv vì:
    - Thay vì phải viết lệnh để di chuyển file, add file vừa di chuyển vào git, và bỏ file ở vị trí cũ ra khỏi git, ta chỉ cần viết 1 lệnh.
  - release/lab01 có là baseline, phiên bản release, không thay đổi.
  - Master phải luôn ổn định, không được sai/hỏng. Cho phép dễ dàng review các thay đổi ở các branch. Dễ dàng revert các thay đổi.

Lab02

- **Thảo luận 2:** Vì sao phải đồng bộ master trước khi PR? Khi nào không được rebase? feature/* khác topic/* ra sao?
  - Vì sao phải đồng bộ master trước khi PR?
    - Vì nhánh search_by_item chậm hơn master, nên khi cài đặt không biết liệu code mới có phù hợp với master hay không.
    - Vậy, đồng bộ trước cho phép phát hiện lỗi sớm hơn, giúp reviewer dễ nhìn ra thay đổi hơn.
  - Khi nào không được rebase? Rebase viết lại lịch sử, tức là đổi SHA của các commit. Vì vậy không rebase khi:
    - Commit đã được chia sẻ và người khác đang dựa vào chúng.
    - Nhánh có nhiều người cùng làm.
    - Trên master và release/*, vì đây là baseline, không được đổi lịch sử.
  - feature/* khác topic/* ra sao?

        - Feature:
            - Thay đổi hành vi sản phẩm mà người dùng thấy được
        - Topic:
            - Chủ đề không phải tính năng: tài liệu, template, cấu hình, sơ đồ,...

Lab03

- **Thảo luận 3:** Merge base là gì? Vì sao master/release/* phải dùng revert?
    - Merge base là gì?
        - Merge base là commit tổ tiên chung gần nhất của hai nhánh, xem bằng git merge-base A B.
        - Khi merge, Git dùng 3-way merge: so sánh base→A và base→B.
            - Chỉ một bên sửa một vùng: lấy bên đó, không xung đột.
            - Cả hai bên sửa cùng vùng theo cách khác nhau: phát sinh xung đột.
    - Vì sao master và release/* phải dùng revert?
        - Với revert thì:
            - Lịch sử vẫn nguyên, ai cũng git pull bình thường.
            - Đi qua nhánh và PR có review, đúng quy trình kiểm soát thay đổi
            - Có thể hoàn tác lại chính bản revert nếu cần.
        - Với master, release, revert giúp:
            - Là baseline
            - Tạo lịch sử để có thể dễ dàng revert cái revert này nếu có lỗi
            - Người khác đã có commit đó rồi. Nếu reset rồi đẩy lên, nhánh được đẩy lên và nhành người khác sẽ lệch nhau, buộc force push.

Lab04:

- **Thảo luận 4:** Vì sao release/* không merge vào master? So sánh với GitFlow. Vì sao lên 1.0.0 mà không phải 0.4.0?
    - Master là nguồn sự thật. Mọi thay đổi vào master đi qua nhánh ngắn hạn và PR. Release chỉ là ảnh chụp đóng băng cắt từ master.
    - Giữ baseline ổn định. Nếu release nhận thay đổi từ nhiều phía, ta khó khẳng định bản đã phát hành chứa chính xác những gì (ảnh hưởng đến kiểm toán cấu hình).

Lab05

- **Vì sao ở Phần A chưa gắn tag v1.1.0? Nếu đã gắn rồi thì bây giờ phải phát hành v1.1.1**
    - Tag không bao giờ được di chuyển. Tag là định danh chính xác một phiên bản. Một khi đã đẩy lên remote và có người dùng, tag phải mãi mãi trỏ vào đúng một commit
    - Chỉ gắn tag khi baseline đã chứa mọi thứ cần có. Phải sửa lỗi trước khi gắn tag.
- **Thảo luận 5:** Ưu, nhược điểm của upstream first so với release first? Nếu quên lan truyền bản sửa sang release/lab05, lệnh nào ở Phần F sẽ phát hiện?
    - Ưu, nhược điểm của upstream first so với release first?
        - Upstream first: sửa ở master trước, rồi cherry-pick xuống các nhánh release.
            - Ưu: Không thể quên sửa ở master, nên bản phát hành sau không tái phát lỗi. Bản sửa được review và test trên mã mới nhất. Chỉ có một nguồn sự thật.
            - Nhược: Chậm hơn khi khẩn cấp, vì phải qua master trước. Master đã đi xa hơn bản cũ nên cherry-pick xuống release có thể xung đột. Bản sửa có thể phụ thuộc vào thay đổi mới mà release cũ không có.
        - Release first: sửa trên nhánh release trước, rồi lan truyền sang master và các release khác.
            - Ưu: Nhanh, sửa và kiểm chứng ngay trên đúng mã đang bị lỗi. Phát hành bản vá sớm. Bản sửa không bị lẫn thay đổi mới của master.
            - Nhược: Rất dễ quên lan truyền sang master hoặc release khác, khiến lỗi quay lại ở bản sau. Vì vậy bắt buộc phải có bước kiểm toán (Phần F).
    - Nếu quên lan truyền sang release/lab05, lệnh nào ở Phần F phát hiện?
        - git log --all --oneline --grep "cherry picked from commit <FIX_SHA>"